

opensearch-project / opensearch-neural-sparse-encoding-v1

like 15
 Follow
 opensearch-project 73

Feature Extraction
 sentence-transformers
 PyTorch
 Safetensors
 Transformers

English
 bert
 fill-mask
 learned sparse
 opensearch
 retrieval
 passage-retrieval

query-expansion
 document-expansion
 bag-of-words
 sparse-encoder
 sparse
 splade

text-embeddings-inference
 arxiv:2411.04403
 License: apache-2.0

Deploy
 Copy to bucket **NEW**
 Use this model

Model card
 Files
 xet
 Community 5

opensearch-neural-sparse-encoding-v1

Select the model

The model should be selected considering search relevance, model inference and retrieval efficiency(FLOPS). We benchmark models' **zero-shot performance** on a subset of BEIR benchmark: TrecCovid,NFCorpus,NQ,HotpotQA,FiQA,ArguAna,Touche,DBPedia,SCIDOCS,FEVER,Climate FEVER,SciFact,Quora.

Overall, the v2 series of models have better search relevance, efficiency and inference speed than the v1 series. The specific advantages and disadvantages may vary across different datasets.

Model	Inference-free for Retrieval	Model Parameters	AVG NDCG@10	FLOPS
opensearch-neural-sparse-encoding-v1		133M	0.524	1
opensearch-neural-sparse-encoding-v2-distill		67M	0.528	8
opensearch-neural-sparse-	✓	133M	0.490	2

Downloads last month
1,150

Safetensors

Model size 0.1B pa

Tensor type 164 · F

Inference Providers **NEW**

Feature Extraction

Your prompt here...

Generate

View Code

Maximize

Model tree for opensearch-projec...

Quantizations 1 model

Paper for opensearch-project/op...

Towards Competitive Search Releva...

Paper • 2411.04403 • Publis... • 2

Run 15,000+ Models Instantly

Inference Providers let you run inference on thousands of models served by our partners using a simple, unified, OpenAI-compatible serverless API ([Learn more](#)).

opensearch-project/opensearch-neural-sparse-encoding-v1 is supported by the following Inference Providers:

HF Inference API

[View API Code](#)

Dismiss

Model	Inference-free for Retrieval	Model Parameters	AVG NDCG@10	F
encoding-doc-v1				
opensearch-neural-sparse-encoding-doc-v2-distill	✓	67M	0.504	1
opensearch-neural-sparse-encoding-doc-v2-mini	✓	23M	0.497	1
opensearch-neural-sparse-encoding-doc-v3-distill	✓	67M	0.517	1
opensearch-neural-sparse-encoding-doc-v3-gte	✓	133M	0.546	1

Overview

- **Paper:** [Towards Competitive Search Relevance For Inference-Free Learned Sparse Retrievers](#)
- **Fine-tuning sample:** [opensearch-sparse-model-tuning-sample](#)

This is a learned sparse retrieval model. It encodes the queries and documents to 30522 dimensional **sparse vectors**. The non-zero dimension index means the corresponding token in the vocabulary, and the weight means the importance of the token.

This model is trained on MS MARCO dataset.

OpenSearch neural sparse feature supports learned sparse retrieval with lucene inverted index. Link: <https://opensearch.org/docs/latest/query-dsl/specialized/neural-sparse/>. The indexing and

search can be performed with OpenSearch high-level API.

Usage (Sentence Transformers)

First install the Sentence Transformers library:

```
pip install -U sentence-transformers
```

Then you can load this model and run inference.

```
from sentence_transformers.sparse_encoder import SparseEncoder

# Download from the 🤗 Hub
model = SparseEncoder("opensearch-project/opensearch-sentence-transformers")

query = "What's the weather in ny now?"
document = "Currently New York is rainy."

query_embed = model.encode_query(query)
document_embed = model.encode_document(document)

sim = model.similarity(query_embed, document_embed)
print(f"Similarity: {sim}")
# Similarity: tensor([[22.3299]])

decoded_query = model.decode(query_embed)
decoded_document = model.decode(document_embed)

for i in range(len(decoded_query)):
    query_token, query_score = decoded_query[i]
    doc_score = next((score for token, score in decoded_document.items() if token == query_token), 0)
    print(f"Token: {query_token}, Query score: {query_score}, Document score: {doc_score}")

# Token: ny, Query score: 2.9262, Document score: 2.5206
# Token: weather, Query score: 2.5206, Document score: 2.0373
# Token: york, Query score: 2.0373, Document score: 1.5786
# Token: cool, Query score: 1.5786, Document score: 1.4636
# Token: current, Query score: 1.4636, Document score: 0.7761
# Token: season, Query score: 0.7761, Document score: 0.7560
# Token: 2020, Query score: 0.7560, Document score: 0.7222
# Token: summer, Query score: 0.7222, Document score: 0.6888
# Token: nina, Query score: 0.6888, Document score: 0.6451
# Token: storm, Query score: 0.6451, Document score: 0.4698
# Token: brooklyn, Query score: 0.4698, Document score: 0.4562
# Token: julian, Query score: 0.4562, Document score: 0.3484
# Token: wow, Query score: 0.3484, Document score: 0.3439
# Token: usa, Query score: 0.3439, Document score: 0.2751
# Token: manhattan, Query score: 0.2751, Document score: 0.2013
# Token: fog, Query score: 0.2013, Document score: 0.1989
# Token: mood, Query score: 0.1989, Document score: 0.1653
# Token: climate, Query score: 0.1653, Document score: 0.1191
# Token: nature, Query score: 0.1191, Document score: 0.0989
```

```
# Token: temperature, Query score: 0.0665, Document score: 0.0665
# Token: windy, Query score: 0.0552, Document score: 0.0552
```

Usage (HuggingFace)

This model is supposed to run inside OpenSearch cluster. But you can also use it outside the cluster, with HuggingFace models API.

```
import itertools
import torch
from transformers import AutoModelForMaskedLM, AutoTokenizer

# get sparse vector from dense vectors with attention weights
def get_sparse_vector(feature, output):
    values, _ = torch.max(output*feature["attention_weights"], dim=-1)
    values = torch.log(1 + torch.relu(values))
    values[:,special_token_ids] = 0
    return values

# transform the sparse vector to a dict of (sample_indices, token_indices)
def transform_sparse_vector_to_dict(sparse_vector, sample_indices, token_indices=torch.nonzero):
    non_zero_values = sparse_vector[(sample_indices, token_indices)]
    number_of_tokens_for_each_sample = torch.sum(non_zero_values, dim=-1)
    tokens = [transform_sparse_vector_to_dict(sparse_vector[sample_index], token_indices)
               for sample_index in range(number_of_tokens_for_each_sample[sample_index])]

    output = []
    end_idx = list(itertools.accumulate([0]+number_of_tokens_for_each_sample.tolist()))
    for i in range(len(end_idx)-1):
        token_strings = tokens[end_idx[i]:end_idx[i+1]]
        weights = non_zero_values[end_idx[i]:end_idx[i+1]]
        output.append(dict(zip(token_strings, weights)))
    return output

# load the model
model = AutoModelForMaskedLM.from_pretrained("openai/gpt2")
tokenizer = AutoTokenizer.from_pretrained("openai/gpt2")

# set the special tokens and id_to_token transform
special_token_ids = [tokenizer.vocab[token] for token in ["[CLS]", "[MASK]", "[SEP]"]]
get_sparse_vector.special_token_ids = special_token_ids
id_to_token = [{"i": i, "token": token} for i in range(tokenizer.vocab.get_vocab_size('tokens'))]
for token, _id in tokenizer.vocab.items():
    id_to_token[_id] = token
transform_sparse_vector_to_dict.id_to_token = id_to_token

query = "What's the weather in ny now?"
document = "Currently New York is rainy."
```

```

# encode the query & document
feature = tokenizer([query, document], padding='max_length', max_length=EMBEDDING_DIM)
output = model(**feature)[0]
sparse_vector = get_sparse_vector(feature, output)

# get similarity score
sim_score = torch.matmul(sparse_vector[0], sparse_vector[1])
print(sim_score)  # tensor(22.3299, grad_fn=<FunctionBackward0>)

query_token_weight, document_query_token_weight = model.module.encoder.get_weights()
for token in sorted(query_token_weight, key=query_token_weight.get):
    if token in document_query_token_weight:
        print("score in query: %.4f, score in document: %.4f" % (query_token_weight[token], document_query_token_weight[token]))

# result:
# score in query: 2.9262, score in document: 2.9262
# score in query: 2.5206, score in document: 2.5206
# score in query: 2.0373, score in document: 2.0373
# score in query: 1.5786, score in document: 1.5786
# score in query: 1.4636, score in document: 1.4636
# score in query: 0.7761, score in document: 0.7761
# score in query: 0.7560, score in document: 0.7560
# score in query: 0.7222, score in document: 0.7222
# score in query: 0.6888, score in document: 0.6888
# score in query: 0.6451, score in document: 0.6451
# score in query: 0.4698, score in document: 0.4698
# score in query: 0.4562, score in document: 0.4562
# score in query: 0.3484, score in document: 0.3484
# score in query: 0.3439, score in document: 0.3439
# score in query: 0.2751, score in document: 0.2751
# score in query: 0.2013, score in document: 0.2013
# score in query: 0.1989, score in document: 0.1989
# score in query: 0.1653, score in document: 0.1653
# score in query: 0.1191, score in document: 0.1191
# score in query: 0.0665, score in document: 0.0665
# score in query: 0.0552, score in document: 0.0552

```

The above code sample shows an example of neural sparse search. Although there is no overlap token in original query and document, but this model performs a good match.

Detailed Search Relevance

Model	Average	Trec Covid	NFCorpus	NQ	HotpotQA
opensearch-neural	0.524	0.771	0.360	0.553	0.600

Model	Average	Trec Covid	NFCorpus	NQ	Ho
sparse-encoding-v1					
opensearch-neural-sparse-encoding-v2-distill	0.528	0.775	0.347	0.561	0.6
opensearch-neural-sparse-encoding-doc-v1	0.490	0.707	0.352	0.521	0.6
opensearch-neural-sparse-encoding-doc-v2-distill	0.504	0.690	0.343	0.528	0.6
opensearch-neural-sparse-encoding-doc-v2-mini	0.497	0.709	0.336	0.510	0.6
opensearch-neural-sparse-encoding-doc-v3-distill	0.517	0.724	0.345	0.544	0.6
opensearch-neural-sparse-encoding-doc-v3-gte	0.546	0.734	0.360	0.582	0.7

License

 System theme

[TOS](#)

[Privacy](#)

[About](#)

[Careers](#)



[Models](#)

[Datasets](#)

[Spaces](#)

[Pricing](#)

[Docs](#)

This project is licensed under the [Apache v2.0](#)

[License](#).

Copyright

Copyright OpenSearch Contributors. See [NOTICE](#) for details.